create_pin_constraint

NAME

create_pin_constraint
Creates individual and bundle pin constraints.

SYNTAX

```
status create_pin_constraint
       -type individual | bundle
       [-cells cells]
       [-layers layers]
       [-pin_spacing_tracks spacing]
       [-pin_spacing_distance distance]
       [-allow_feedthroughs true | false]
       [-sides side_numbers]
       [-width width]
       [-length length]
       [-nets nets]
       [-pins pins]
       [-ports ports]
       [-exclude_sides exclude_side_numbers]
       [-offset offset_distance_or_pct]
       [-off_edge]
       [-location {location}]

       [-bundles bundles]
       [-keep_pins_together true | false]
       [-range {range_distance_or_pct}]
       [-bundle_order ordered | increasing | decreasing | equal-distance]
       [-interleaving_layer_range layer_range]
       [-interleaving_bit_offset bit_offset]
       [-self]
```

Data Types

cells	collection
layers	collection
spacing	integer
distance	distance
side_numbers	collection
width	string
length	string
nets	collection
pins	collection
ports	collection
exclude_side_numbers	collection
offset_distance	string
location	float
bundles	collection
range	string
layer_range	integer
bit_offset	integer

ARGUMENTS

Note the first set of arguments applies to both individual and bundle pin constraints. The second set of arguments applies to only individual pin constraints, and the third set of arguments applies only to bundle pin constraints. If you provide a type-specific argument for the incorrect type, the command returns an error message.

-type individual | bundle
Specifies the type of pin constraint to create.

-cells cell_collection
Limits the constraints to the specified cells. The cells must correspond to sub levels of physical hierarchy in the top design. When creating individual pin constraints, this argument is only valid when used with -nets.

-layers metal_layers
Specifies the set of metal layers allowed for pin placement. The argument is a list of metal layer names as returned by the get_layers command.

By default, the metal layers from M2 to the maximum metal layer (specified earlier during floorplan flow) are allowed for pin placement. If the maximum metal layer has not been specified, the maximum available metal layer specified in the technology

file is used as the default. If the maximum layer specified is beyond the maximum metal layer for the current design, then the maximum layer in the technology file is used.

-pin_spacing_tracks spacing_number
Specifies the minimum number of wire tracks between adjacent pins or between a pin and a preroute wire that cuts across a block edge in the normal direction. Pins that are created are always snapped to wire tracks. The default pin-to-pin spacing is one wire track. The spacing number must be 0 or a positive integer.

This option is mutually exclusive with -pin_spacing_distance.

-pin_spacing_distance spacing_distance
Specifies the minimum distance between adjacent pins or between a pin and a preroute wire that cuts across a block edge in the normal direction. Pins that are created are always snapped to wire tracks.

This option is mutually exclusive with -pin_spacing_tracks.

-allow_feedthroughs true | false
Specifies whether feedthrough ports can be created in pin placement flow. When true, the global router can create feedthrough routing on a block cells for the specified nets. When a feedthrough is created on a top-level net, the net is split into a set of new top-level nets and child-level nets if feedthrough nets are created for the child nets. Directions are assigned for the new feedthrough ports, and the netlist is modified to accommodate the new ports in the block cell.

When used for individual pin constraints, this option can only be specified together with the -nets option and cannot be used together with -cells option. To set feedthrough constraints on individual block cells for selected nets, create a pin constraints file and use the read_pin_constraints command.

This option can also be used to set feedthrough allowance for the push-down into a block of pre routed nets using push_down_objects. That command checks this pin constraint, and if found to be set, and set to true, creates feedthroughs based upon the pre route multiple crossing of a block's boundary.

For individual pin constraints, this option is only applicable with the -net argument. Also, for individual pin constraints, this option is mutually exclusive with the -cells argument.

-sides side_numbers
Specifies one or more block sides on which the pin must be placed.

A side number is a positive integer that starts at one, and increments by one as you proceed clockwise around each edge in the shape. Given any rectilinear shape, the left-most vertical edge is the starting edge (side number 1). If there are multiple left-most edges, then the edge 1 is the lowest left-most edge. You cannot specify a value of 0 for this option.

For individual pin constraints, this option is mutually exclusive with -exclude_sides option.

-width pin_width
Specifies the width of the pin. The width is perpendicular to the layer's preferred routing direction. The syntax can be a single real number to indicate the width is applied to all allowed layers (referred to as common pin width). Or alternatively, the width can be specified as a list of layer-real pairs as the following (referred to as per layer pin width):

-width {{M2 0.2} {M3 0.2} {M4 0.3} {M5 0.3}}

It means that if the pin is to be placed on layer M2 or M3, its width should be 0.2 and if on M4 or M5, 0.3. On other layers, use the default width defined by the tech file.

If you first set the pin width to be common pin width, then set the pin width to be per layer width with a second create_pin_constraint command, then the width specified in the second command overrides the first. Likewise, if you first set the pin width to be per-layer width, then set the pin width to be a common pin width with a second create_pin_constraint command,

then the width specified in the second command overrides the first.

However, if you first set the pin width to be per layer, then in a second command sets the pin width again to be per layer but with different layers or different values, then the combined per layer pin width is the final constraints. For example, consider the following example:

```
create_pin_constraint -bundles {bundle1} -width 0.3
create_pin_constraint -bundles {bundle1} -width {{M2 0.3} {M3 0.3}}
```

The first common pin width constraint is overridden by the second per layer pin width. The pin width should be 0.3 on layer M2 or M3, but default width on all other layers.

And for the following example:

```
create_pin_constraint -bundles {bundle1} -width {{M3 0.4} {M4 0.4}}
create_pin_constraint -bundles {bundle1} -width {{M2 0.3} {M3 0.3}}
```

The pin width should be 0.3 on layer M2 or M3, 0.4 on layer M4, and default width on all other layers.

-length pin_length
: Specifies the length of the pin. The length is in parallel to the layer's preferred routing direction. The syntax can be a single real number to indicate the length is applied to all allowed layers (referred to as common pin length). Or alternatively, the length can be specified as a list of layer-real pairs as the following (referred to as per layer pin length):

  ```
  -length {{M2 0.2} {M3 0.2} {M4 0.3} {M5 0.3}}
  ```

  It means that if the pin is to be placed on layer M2 or M3, its length should be 0.2 and if on M4 or M5, 0.3. On unspecified layers, the pin length is derived from the pin width. See the man page of place_pins for details.

-nets nets
: Applies the constraints only to the nets specified in nets.

-pins pins
: Specifies the pins that receive the constraints. Specify the full pin name from top-level design.

-ports ports
: Specifies the ports that receive the constraints.

-exclude_sides exclude_side_numbers
: Specifies the block edges on which pins cannot be placed. The exclude_side_numbers argument is a list of positive integers.

  A side number is a positive integer that starts at one, and increments by one as you proceed clockwise around each edge in the shape. Given any rectilinear shape, the left-most vertical edge is the starting edge (side number 1). If there are multiple left-most edges, then the edge 1 is the lowest left-most edge. You cannot specify a value of 0 for this option.

  When creating individual pin constraints, this option is mutually exclusive with -sides option.

-offset offset_distance
: Specifies the distance in microns between the starting or ending point of a specified edge and the pin's center location. The starting point is the location where the edge begins as you proceed clockwise around the shape. The ending point is the location where the edge end as you proceed clockwise around the shape. The starting point of edge 1 is the lowest point of the edge, the ending point of edge 1 is the highest point of the edge the starting point of edge 2 is the leftmost point of the edge, and so on. Positive value means offset distance calculates from starting point of a specified edge and the pin's center location, vice versa for negative value. Value of offset can be either a distance or a percentage. To specify a percentage, append a '%' character to the end of the float value.

  A range can also be specified for this option. Specify range start and range end values within which the pins are allowed to place. The start and end values refer to the distances as measured to the starting point of the edge in microns (or the

default unit). Negative value for range refer to the distances as measured to the ending point of the edge in microns (or the default unit).Value of offset range can be either a distance or a percentage.To specify a percentage, append a '%' character to the end of the float value. For example,

-offset {10 40}

-offset {10% 40%}

-offset {10% 40}

You must specify the side information together with the offset. However, offset cannot be specified with multiple sides.

-off_edge

Specifies whether the pin is created on the block boundary. This option is mutually exclusive with the -sides and -offset options.

-location location

Specifies the x- and y-location in the top-level design where the pin should be created. This option is mutually exclusive with the -sides and -offset options. When used together with the -off_edge option, the pin is placed on the closest wiretrack to the specified location. Note that in this case there is no legalization of pin placement for off edge pins, as it is meant to be a basic placement where you need to give the legal location. If the location constraint is a hard constraint set by the create_block_pin_constraint command, the center of the pin is placed at the exact location specified by this option. If you do not specify the -off_edge option, the pins are placed on the edge that is closest to this location.

-bundles bundles

Limits the constraints to the specified net bundles. The bundles must be in the top-level of the current design. If not specified, the command applies the constraints to all net bundles.

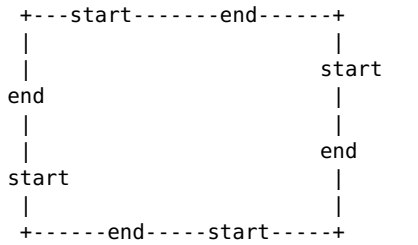
-keep_pins_together true | false

Specify whether bundle pins are kept together. When this option is true, all bundle pins on the same block are placed on the same layer (hard constraint) and no other pins should be placed between the bundle pins (soft constraint). In certain cases, the command might violate the soft constraint and insert other pins between the bundle pins. The default is false.

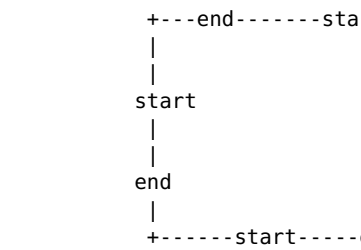
-range {range}

Specifies the range of positions on a side within which the bundle pins must be placed. The positive start and end values refer to the distances as measured to the starting point of the edge in microns (or the default unit). The negative start and end values refer to the distances as measured to the ending point of the edge in microns (or the default unit). Positive start is the position that is closest to the edge's starting point, and positive end is the position that is farthest to the edge's starting point. Vice versa for negative start and end positions. The starting point of an edge is the vertex of the edge where the edge begins as you proceed clockwise around the block's boundary from the lowest vertex of the leftmost edge. If there are more than one leftmost edges, then from the lowest one. Value of range can be either a distance or a percentage. To specify a percentage, append a '%' character to the end of the float value.

The following simple drawing shows an example of how positive start and end are measured on a rectangular block on each of the sides.



The following simple drawing shows an example of how negative start and end are measured on a rectangular block on each of the sides.



-bundle_order ordered | increasing | decreasing | equal-distance
Specifies the relative placement order of the pins connected to the bundle nets. Note that the order refers to the net order in the bundle, not the pins on the block. The bundle order is measured as seen at the current top design. Pin placement considers this option only when option -keep_pins_together is set to true. If two or more bundles connecting the same collection of multiple instantiated blocks pins but with conflict orders, only one of them can be honored.

o ordered: Pins are ordered according to the net order in the bundle. For vertical block sides, bundle pins can be placed either from bottom to top or from top to bottom. For horizontal sides, bundle pins can be placed either from left to right or from right to left. The orders on each edges are chosen such that for the bundle nets connect the bundle pins from one edge to another edge, there is no net crossing, that is river turn style.

In the following example, bundle pin constraint is specified as ordered so that the nets connecting two bundle pins are of the river turn style, that is no crossing.

```
prompt> create_pin_constraint -type bundle -layers {M3} -bundles bundle1 \
   -bundle_order ordered -keep_pins_together true
```

o increasing: Pins are ordered according to the increasing net order in the bundle. For vertical block sides, bundle pins are placed from bottom to top. For horizontal block sides, bundle pins are placed from left to right.

In the following example, bundle pin constraint is specified on a rectangular block so that bundle pins are placed on side 2 (horizontal edge) of block. Pins are placed from left to right (Least significant bit is the leftmost; Most significant bit is the right most on side2).

```
prompt> create_pin_constraint -type bundle -bundles bundle1 -sides 2 \
   -bundle_order increasing -keep_pins_together true
```

o decreasing: Pins are ordered according to the decreasing net order in the bundle. For vertical block sides, bundle pins are placed from top to bottom. For horizontal block sides, bundle pins are placed from right to left.

In the following example, bundle pin constraint is specified on a rectangular block so that bundle pins are placed on side 2 (horizontal edge) of block. Pins are placed from right to left (Most significant bit is the leftmost; Least significant bit is the rightmost).

```
prompt> create_pin_constraint -type bundle -bundles bundle1 -sides 2 \
   -bundle_order decreasing -keep_pins_together true
```

o equal-distance: Pins are ordered as with -bundle_order ordered, but the pin ordering for two edges is chosen to create the same Manhattan distance between each corresponding pair of pins.

In the following example, bundle pin constraint is specified as equal-distance so that the Manhattan distances of each bits of the bundle are the same for bundle bundle1.

```
prompt> create_pin_constraint -type bundle -layers {M3} -bundles bundle1 \
   -bundle_order equal-distance -keep_pins_together true
```

Note that when setting bundle pin constraints on pins of multiple instantiated blocks, as the orientation of multiple instantiated blocks can be different, the tool chooses one of the multiple instantiated blocks randomly to honor the constraints. Therefore the bundle constraints could be not hon-

ored for the other multiple instantiated blocks.

When option -keep_pins_together is set to true but option -bundle_order is not specified, the tool chooses from either increasing or decreasing based on alignment, abutment and wire length cost. The bundle order might also be decided by already placed or fixed bundle pins on this bundle due to alignment or abutment.

-interleaving_layer_range layer_range
Enables the folding bundle pin placement when layer_range value is non zero and the option -keep_pins_together is true.

Folding bundle pin placement means to keep the bundle pins together and on multiple layers. The multiple layers used in folding bundle pin placement are the layers specified by the option -layers, and N upper layers of the same routing direction, where N is specified by layer_range.

For example, if the constraints are set as

-keep_pins_together true -layers metal2 -interleaving_layer_range 2

Then the bundle pins are placed on layers metal2, metal4 and metal6. The bit-0 pin is placed on metal2, bit-1 pin is placed on metal4, bit-2 pin is placed on metal6. Then bit-3 pin back on metal2, bit-4 pin on metal4, bit-5 pin on metal6. So on and so forth until all of the bundle pins are placed. On each of the layer, the subset of the bundle pins, for example, bit-0, bit-3, ..., on metal2, are all kept together honoring the spacing constraint specified by -pin_spacing_tracks or -pin_spacing_distance.

This option does not honor the option -cells. After setting, it applies to all the bundle pins if option -bundles is not used, or all the pins on the specified bundles by option -bundles.

If the option -keep_pins_together is false, this option is ignored.

-interleaving_bit_offset bit_offset
Specifies the relative offset between the first pins on the upper and lower layers in folding bundle pin placement for the pins of the same bundle on the same cell or the top ports of the same bundle.

The default is 0, meaning no bit offset between the first pins on the upper and lower layers. The first pin on the upper layer is on the closest track following the first pin on the immediate lower layer.

If the bit_offset is set a non zero value N, then the first pin on the upper layer leaves N empty tracks following the first pin on the immediate lower layer.

This option only affects the first pins' relative locations on the multiple layers in the folding bundle pin placement. The remaining pins relative locations are solely determined by the spacing constraints.

This option does not honor the option -cells. After setting, it applies to all the bundle pins if option -bundles is not used, or all the pins on the specified bundles by option -bundles.

If the option -keep_pins_together is false, this option is ignored.

-self Applies the constraints to the top-level block. You can combine the -cells and -self options. If both the -cells and -self options are unspecified, the constraints apply to all block cells.

DESCRIPTION
This command constrains the placement of net bundles, individual pins, individual ports, or individual nets on a physical design block. The pin placement constraints are saved in the design database. The constraints are honored by the place_pins command.

The create_pin_constraint command is additive. The constraint values set in previous calls are retained, unless they are explicitly overridden by subsequent calls.

The set_editability command can enable or disable the create_pin_constraint command for specified blocks or levels of physical hierarchy. This command has no effect on the blocks that are disabled with the set_editability command.

o Global constraints: Constraints apply to all bundles and all block instances.

o Specific bundle constraints: Constraints apply to a specific bundle and all block instances.

o Specific block constraints: Constraints apply to all bundles and a specific block instance. Note that this can apply to the top-level block when you specify the -self option.

o Bundle and block pair constraints: Constraints apply to a specific bundle and a specific block instance. Note that this can apply to the top-level block when you specify the -self option.

Upon success, the command returns a collection of pin constraints containing the newly created pin constraint.

EXAMPLES

The following example sets the allowed pin layers to METAL2 and METAL3 for bundle BUNDLE_0. This creates a specific bundle constraint.

```
prompt> create_pin_constraint -type bundle -layers [get_layers METAL2 \
   METAL3] -bundles [get_bundles BUNDLE_0]
```

The following example enforces ordering on all bundle pins of cell BLOCK1. This creates a specific block constraint.

```
prompt> create_pin_constraint -type bundle -bundle_order ordered \
   -cells [get_cells BLOCK0] -keep_pins_together true
```

The following example enforces ordering and minimum pin spacing for bundle BUNDLE_0. This creates a specific bundle constraint.

```
prompt> create_pin_constraint -type bundle -bundle_order ordered \
   -pin_spacing_tracks 2 -bundles [get_bundles BUNDLE_0] -keep_pins_together true
```

The following example sets the allowed pin layers for net clk to M2 and M3.

```
prompt> create_pin_constraint -type individual -layers [get_layers {M2 M3}] \
   -nets [get_nets clk]
```

The following example sets the allowed pin layers for net clk to M2 and M3 having start offset as 10% of side 3 and endOffset as 40% of side 3.

```
prompt> create_pin_constraint -type individual -layers [get_layers {M2 M3}] \
   -nets [get_nets clk] -offset {10% 40%} -side 3
```

SEE ALSO

place_pins(2)
push_down_objects(2)
report_pin_constraints(2)
remove_pin_constraints(2)
get_pin_constraints(2)
create_block_pin_constraint(2)
read_pin_constraints(2)
set_editability(2)
